1) Цель

Реализовать ключевые метрики классификации вручную и применить несколько алгоритмов на реальных данных с дисбалансом классов.

2) Задание

Практические задания приведены в Jupyter-ноутбуке exercises.ipynb.

Рекомендуется выполнять задания по очереди.

3) Результат выполнения

В результате были выполнены все обязательные задания и одно бонусное задание на 1 балл. Задание прошло автоматическое тестирование на 9/10

4) Исходный код

exercises.ipynb

```
# # Лабораторная работа №3
# ## Классификация и оценка качества моделей
#
# **Курс:** Основы машинного обучения
# **Цель:** Реализовать ключевые метрики классификации вручную и применить несколько алгоритмов на реальных данных с дисбалансом классов.
# **Тема:** Классификация и оценка качества моделей
# **Дедлайн:** см. таблицу учёта успеваемости
#
# ---
#
# ### Инструкция по выполнению
#
# 1. **Вариант:** укажите свой `Student_ID` в задании 0.1 — он определит датасет и параметры задания
# 2. **Именование переменных:** строго соблюдайте указанные имена — они используются для автопроверки
# 3. **Функции:** реализуйте каждую функцию с указанной сигнатурой; не меняйте имена функций и параметров
# 4. **Визуализации:** все графики должны иметь заголовок и подписи осей
# 5. **Сохранение:** перед сдачей выполните `Kernel → Restart & Run All` и убедитесь, что ошибок нет



# ---
# ## Блок 0: Настройка варианта
```

```
# ### Задание 0.1: Укажите свой Student_ID
#
# Найдите свой порядковый номер (поле «№ п/п») в таблице учёта
успеваемости курса и укажите его ниже.


Student_ID = 7

# ### Задание 0.2: Определение варианта
#
# Выполните ячейку ниже — она определит ваш датасет и параметры задания.


# (dataset_name, uci_id, target_col, description)
# dataset_name — короткое название датасета
# uci_id       — идентификатор датасета в репозитории UCI ML Repository
# target_col   — название целевой переменной
# description  — полное название датасета и источник для загрузки
datasets = [
    (
        'bank_marketing',
        222,
        'y',
        'Bank Marketing (UCI id=222): подписка клиента на срочный вклад, '
        'https://archive.ics.uci.edu/dataset/222/bank+marketing'
    ),
    (
        'credit_default',
        350,
        'default payment next month',
        'Default of Credit Card Clients (UCI id=350): дефолт по кредитной карте, '
        'https://archive.ics.uci.edu/dataset/350/default+of+credit+card+clients'
    ),
    (
        'htru2',
        372,
        'class',
        'HTRU2 (UCI id=372): классификация кандидатов в пульсары, '
        'https://archive.ics.uci.edu/dataset/372/htru2'
    ),
```

```
    (
        'adult',
        2,
        'income',
        'Adult / Census Income (UCI id=2): доход физического лица выше $50K, '
        'https://archive.ics.uci.edu/dataset/2/adult'
    ),
    (
        'online_shoppers',
        468,
        'Revenue',
        'Online Shoppers Purchasing Intention (UCI id=468): намерение совершить
        покупку, '
        'https://archive.ics.uci.edu/dataset/468/online+shoppers+purchasing+intentio
        n+dataset'
    ),
]

if Student_ID is None:
    print("ОШИБКА! Не указан Student_ID. Вернитесь к заданию 0.1.")
else:
    dataset_id = (Student_ID - 1) % len(datasets)
    DATASET_NAME, UCI_ID, TARGET_COL, DATASET_DESC =
    datasets[dataset_id]

    print(f"Ваш вариант: датасет №{dataset_id + 1}")
    print(f"Датасет:     {DATASET_DESC}")
    print(f"Целевая переменная: {TARGET_COL}")

# ### Задание 0.3: Установка зависимостей



# Установка необходимых библиотек (выполните один раз)
!pip install ucimlrepo
!pip install imbalanced-learn  # используется в бонусном задании 4.1

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.figure
from typing import Tuple, Dict
```

```
from ucimlrepo import fetch_ucirepo
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn.utils.validation import check_is_fitted

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

print("Библиотеки загружены.")

# ---
# ## Блок 1: Обработка данных
#
# **Максимум: 20 баллов**



# ### Задание 1.1 — Разведочный анализ данных (5 баллов)
#
# Загрузите датасет с помощью `ucimlrepo` и реализуйте функцию `plot_eda`, которая строит два графика на одной фигуре:
#
# 1. **Распределение целевой переменной** — столбчатый график (bar chart) с абсолютными значениями и долями каждого класса.
# 2. **Тепловая карта корреляций** числовых признаков (`DataFrame.corr()`).
#
# По результатам EDA ответьте на вопросы в переменной `eda_comment`:
# - Есть ли дисбаланс классов? Насколько сильный?
# - Есть ли сильно скоррелированные признаки, которые стоит рассмотреть при предобработке?
#
# > **Подсказка:** `fetch_ucirepo(id=UCI_ID)` возвращает объект с полями `.data.features` и `.data.targets`.



def plot_eda(df: pd.DataFrame, target_col: str) -> matplotlib.figure.Figure:
    """Разведочный анализ датасета.
```

```
    Параметры:
        df         — исходный датафрейм, включающий целевую переменную
        target_col — название столбца с целевой переменной

    Возвращает:
        matplotlib.figure.Figure с двумя подграфиками:
        левый — распределение классов, правый — тепловая карта корреляций
    """
    fig, axes = plt.subplots(1, 2, figsize=(36, 16))

    class_counts = df[target_col].value_counts().sort_index()
    class_labels = class_counts.index.astype(str)
    axes[0].bar(class_labels, class_counts.values)
    axes[0].set_title(f"Распределение классов: {target_col}")
    axes[0].set_xlabel("Класс")
    axes[0].set_ylabel("Количество")

    corr = df.select_dtypes(include=[np.number]).corr()
    heatmap = axes[1].imshow(corr, cmap="coolwarm", vmin=-1, vmax=1,
    aspect="auto")
    axes[1].set_title("Тепловая карта корреляций")
    axes[1].set_xticks(range(len(corr.columns)))
    axes[1].set_yticks(range(len(corr.index)))
    axes[1].set_xticklabels(corr.columns)
    axes[1].set_yticklabels(corr.index)
    fig.colorbar(heatmap, ax=axes[1])

    fig.tight_layout()
    return fig


# Загрузка датасета
dataset = fetch_ucirepo(id=350)
df: pd.DataFrame = pd.concat([dataset.data.features, dataset.data.targets], axis=1)

df = df.rename(columns={"Y": TARGET_COL})

# TODO: вызовите plot_eda и сохраните результат
fig_eda: matplotlib.figure.Figure = plot_eda(df, TARGET_COL)
```

```
# TODO: запишите наблюдения по EDA
eda_comment: str = (
    "Дисбаланс: класс непросроченного платежа заметно преобладает (класс
    просроченного платежа существенно меньше); "
    "Корреляции: наибольшие положительные связи с целевой переменной у
    признаков платежной истории "
    "PAY_0-PAY_6, при этом внутри групп BILL_AMT* и PAY_AMT*
    наблюдается выраженная взаимная корреляция."
)

df.head()

# ### Задание 1.2 — Базовая предобработка (5 баллов)
#
# Создайте датафрейм `df_base`, выполнив **обязательные** шаги
предобработки:
#
# 1. **Обработка пропусков.** Проверьте наличие явных (`NaN`) и неявных
пропусков (например, значения `"unknown"` или `"?"`, нестандартные
числовые коды). Заполните или удалите их обоснованно.
# 2. **Кодирование категориальных признаков.** Все нечисловые столбцы
должны быть преобразованы в числовые (например, с помощью `LabelEncoder`
или `pd.get_dummies`).
#
# **Требования:**
# - В `df_base` не должно остаться `NaN`.
# - Все столбцы должны быть числового типа.
# - Исходный датафрейм `df` **не перезаписывать**.
#
# >  **Подсказка:** для некоторых датасетов пропуски закодированы не как
`NaN`, а как специальные значения — проверьте документацию вашего датасета
по ссылке из задания 0.2.



# Базовая предобработка
_df_base = df.copy()

# Категориальные признаки
categorical_cols = [
    "X2",   # SEX
    "X3",   # EDUCATION
```

```
    "X4",   # MARRIAGE
    "X6",   # PAY_0
    "X7",   # PAY_2
    "X8",   # PAY_3
    "X9",   # PAY_4
    "X10",  # PAY_5
    "X11",  # PAY_6
]

# one-hot
_df_encoded = pd.get_dummies(_df_base, columns=categorical_cols, dtype=int)

ordered_cols = []
for col in _df_base.columns:
    if col in categorical_cols:
        ordered_cols.extend([c for c in _df_encoded.columns if
        c.startswith(f"{col}_")])
    else:
        ordered_cols.append(col)

df_base: pd.DataFrame = _df_encoded[ordered_cols]

df_base.info()

# ### Задание 1.3 — Расширенная предобработка (5 баллов)
#
# На основе `df_base` создайте датафрейм `df_processed`, добавив
**дополнительные** шаги по вашему выбору. Возможные варианты:
#
# - Стандартизация числовых признаков (`StandardScaler`).
# - Удаление выбросов.
# - Удаление сильно скоррелированных признаков (по результатам EDA).
# - Удаление дубликатов.
#
# Обоснуйте каждый шаг в переменной `preprocessing_comment`: почему
именно эти шаги, и как они должны повлиять на качество модели.
#
# > **Подсказка:** `df_processed` должен **отличаться** от `df_base` хотя
бы одним шагом. Пустое обоснование не засчитывается.
```

```
# Расширенная предобработка
# Шаг 1 удаляем дубликаты

print(f"Количество полных дубликатов: {df_base.duplicated().sum()}")
df_processed = df_base.drop_duplicates().copy()

# Шаг 2 стандартизируем числовые признаки
# One-hot и целевой не масштабируем
continuous_cols = ["X1", "X5", "X12", "X13", "X14", "X15", "X16", "X17",
"X18", "X19", "X20", "X21", "X22", "X23"]

scaler = StandardScaler()
df_processed[continuous_cols] =
scaler.fit_transform(df_processed[continuous_cols])

# TODO: обоснуйте каждый шаг предобработки
preprocessing_comment: str = (
    "Шаг 1: удалены дубликаты, потому что повторяющиеся наблюдения могут
    искажать распределение классов и смещать обучение; "
    "Шаг 2: выполнена стандартизация непрерывных признаков (X1, X5,
    X12-X23), потому что признаки имеют сильно разные масштабы "
)

df_processed.head()

# ### Задание 1.4 — Разделение на выборки (5 баллов)
#
# Разделите `df_processed` на три выборки: обучающую, валидационную и
тестовую.
#
# **Требования:**
# - Пропорция разделения: 60% / 20% / 20% или 70% / 15% / 15% — на ваш
выбор.
# - Используйте **стратификацию** по целевой переменной (`stratify=y`),
чтобы сохранить соотношение классов.
# - Зафиксируйте `random_state=RANDOM_STATE`.
#
# Валидационная выборка понадобится в блоке 3 для подбора оптимального
порога классификации — **не используйте её для обучения моделей**.
#
```

```
# > **Подсказка:** разделение на три части удобно делать в два вызова `train_test_split`: сначала отделить тест, затем разделить остаток на обучение и валидацию.


X: pd.DataFrame = df_processed.drop(columns=[TARGET_COL])
y: pd.Series = df_processed[TARGET_COL]

# Разделение 60% / 20% / 20% со стратификацией
# Сначала отделяем тест (20%), затем делим оставшиеся 80% на train/val как 75%/25%
X_temp, X_test, y_temp, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=RANDOM_STATE,
    stratify=y,
)

X_train, X_val, y_train, y_val = train_test_split(
    X_temp,
    y_temp,
    test_size=0.25,
    random_state=RANDOM_STATE,
    stratify=y_temp,
)

# Проверка разбивки
if all(v is not None for v in [X_train, X_val, X_test]):
    total = len(X_train) + len(X_val) + len(X_test)
    print(f"Train: {len(X_train)} ({len(X_train)/total:.1%})")
    print(f"Val:   {len(X_val)}   ({len(X_val)/total:.1%})")
    print(f"Test:  {len(X_test)}  ({len(X_test)/total:.1%})")
    print(f"Итого: {total} (исходный: {len(df_processed)})")

# ---
# ## Блок 2: Ручная реализация метрик
#
# **Максимум: 30 баллов**
#
```

В этом блоке **запрещено** использовать `sklearn.metrics` для вычисления самих метрик. Используйте только `numpy` и `pandas`. После реализации вы сможете проверить себя сравнением с `sklearn`.

Задание 2.1 — Матрица ошибок, точность, полнота, F1 (10 баллов)
#
Реализуйте пять функций бинарной классификации, начиная с матрицы ошибок — все остальные метрики выводятся непосредственно из неё.
#
Матрица ошибок возвращает четыре значения:
$$TP, FP, FN, TN$$
где TP — истинно положительные, FP — ложно положительные, FN — ложно отрицательные, TN — истинно отрицательные.
#
Остальные метрики:
#
$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$
#
$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$
#
$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$
#
Функции `precision_manual`, `recall_manual`, `f1_manual` должны использовать `confusion_matrix_manual` внутри — не дублировать логику подсчёта TP/FP/FN/TN.
#
Все функции должны обрабатывать **деление на ноль** — возвращать `0.0` в случае неопределённости.
#
После реализации проверьте совпадение с `sklearn.metrics` с точностью `1e-6`.

```
def confusion_matrix_manual(
    y_true: np.ndarray,
    y_pred: np.ndarray
) -> Tuple[int, int, int, int]:
    """Элементы матрицы ошибок для бинарной классификации.
```

```
    Параметры:
        y_true — истинные метки классов, массив из 0 и 1
        y_pred — предсказанные метки классов, массив из 0 и 1

    Возвращает:
        Кортеж (TP, FP, FN, TN)
    """
    tp = int(np.sum((y_true == 1) & (y_pred == 1)))
    fp = int(np.sum((y_true == 0) & (y_pred == 1)))
    fn = int(np.sum((y_true == 1) & (y_pred == 0)))
    tn = int(np.sum((y_true == 0) & (y_pred == 0)))

    return tp, fp, fn, tn


def accuracy_manual(y_true: np.ndarray, y_pred: np.ndarray) -> float:
    """Доля правильно классифицированных объектов.

    Параметры:
        y_true — истинные метки классов, массив из 0 и 1
        y_pred — предсказанные метки классов, массив из 0 и 1

    Возвращает:
        float — значение accuracy в диапазоне [0, 1]
    """
    tp, fp, fn, tn = confusion_matrix_manual(y_true, y_pred)
    total = tp + fp + fn + tn
    if total == 0:
        return 0.0
    return (tp + tn) / total


def precision_manual(y_true: np.ndarray, y_pred: np.ndarray) -> float:
    """Точность: доля истинно положительных среди предсказанных
    положительных.

    Параметры:
        y_true — истинные метки классов, массив из 0 и 1
        y_pred — предсказанные метки классов, массив из 0 и 1

    Возвращает:
```

```
    float — значение precision в диапазоне [0, 1]; 0.0 при TP + FP = 0
    """
    tp, fp, _, _ = confusion_matrix_manual(y_true, y_pred)
    denom = tp + fp
    if denom == 0:
        return 0.0
    return tp / denom


def recall_manual(y_true: np.ndarray, y_pred: np.ndarray) -> float:
    """Полнота: доля истинно положительных среди всех реальных
    положительных.

    Параметры:
        y_true — истинные метки классов, массив из 0 и 1
        y_pred — предсказанные метки классов, массив из 0 и 1

    Возвращает:
        float — значение recall в диапазоне [0, 1]; 0.0 при TP + FN = 0
    """
    tp, _, fn, _ = confusion_matrix_manual(y_true, y_pred)
    denom = tp + fn
    if denom == 0:
        return 0.0
    return tp / denom


def f1_manual(y_true: np.ndarray, y_pred: np.ndarray) -> float:
    """F1-мера: гармоническое среднее точности и полноты.

    Параметры:
        y_true — истинные метки классов, массив из 0 и 1
        y_pred — предсказанные метки классов, массив из 0 и 1

    Возвращает:
        float — значение F1 в диапазоне [0, 1]; 0.0 при P + R = 0
    """
    precision = precision_manual(y_true, y_pred)
    recall = recall_manual(y_true, y_pred)
    denom = precision + recall
    if denom == 0:
```

```
        return 0.0
    return 2 * precision * recall / denom


# Проверка реализации относительно sklearn

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
y_demo  = np.array([1, 0, 1, 1, 0, 0, 1, 0])
yp_demo = np.array([1, 0, 0, 1, 0, 1, 1, 0])
print('confusion_matrix_manual:', confusion_matrix_manual(y_demo, yp_demo))
print('sklearn confusion_matrix (TN,FP,FN,TP):', confusion_matrix(y_demo, yp_demo).ravel())
print('Accuracy  — ручная:', accuracy_manual(y_demo, yp_demo),  '| sklearn:', accuracy_score(y_demo, yp_demo))
print('Precision — ручная:', precision_manual(y_demo, yp_demo), '| sklearn:', precision_score(y_demo, yp_demo))
print('Recall    — ручная:', recall_manual(y_demo, yp_demo),    '| sklearn:', recall_score(y_demo, yp_demo))
print('F1        — ручная:', f1_manual(y_demo, yp_demo),        '| sklearn:', f1_score(y_demo, yp_demo))


# ### Задание 2.2 — PR-кривая вручную (8 баллов)
#
# Реализуйте функцию, вычисляющую **PR-кривую** (кривую точность–полнота) путём перебора порогов классификации.
#
# Алгоритм:
# 1. В качестве порогов используйте все уникальные значения `y_score`, отсортированные по убыванию.
# 2. Для каждого порога $t$ получите бинарные предсказания: $\hat{y}_i = 1$ если $s_i \geq t$, иначе $0$.
# 3. Вычислите `precision` и `recall` для каждого порога, используя функции из задания 2.1.
#
# Функция должна возвращать три массива одинаковой длины: `(precision, recall, thresholds)`.


def precision_recall_curve_manual(
```

```
    y_true: np.ndarray,
    y_score: np.ndarray
) -> Tuple[np.ndarray, np.ndarray, np.ndarray]:
    """Вычисление PR-кривой путём перебора порогов.

    Параметры:
        y_true  — истинные метки классов, массив из 0 и 1, размерность (n,)
        y_score — оценки вероятности положительного класса, размерность (n,)

    Возвращает:
        Кортеж (precision, recall, thresholds), каждый — np.ndarray размерности
        (k,),
        где k — число уникальных порогов
    """
    thresholds = np.sort(np.unique(y_score))[::-1]

    precision_values = []
    recall_values = []

    for t in thresholds:
        y_pred = (y_score >= t).astype(int)
        precision_values.append(precision_manual(y_true, y_pred))
        recall_values.append(recall_manual(y_true, y_pred))

    precision = np.asarray(precision_values)
    recall = np.asarray(recall_values)

    return precision, recall, thresholds


# ### Задание 2.3 — ROC-кривая и ROC-AUC вручную (8 баллов)
#
# Реализуйте функции для вычисления **ROC-кривой** и площади под ней
(AUC).
#
# **ROC-кривая** строится в координатах FPR–TPR при переборе порогов:
#
# $$\text{TPR} = \frac{TP}{TP + FN}, \quad \text{FPR} = \frac{FP}{FP +
TN}$$
#
# **AUC** вычисляется численным интегрированием методом трапеций:
#
```

```
# $$\text{AUC} = \int_0^1 \text{TPR}(\text{FPR})\, d(\text{FPR}) \approx
\sum_{i=1}^{k-1} \frac{\text{TPR}_i + \text{TPR}_{i+1}}{2} \cdot
|\text{FPR}_{i+1} - \text{FPR}_i|$$
#
# ROC-кривая должна начинаться в точке (0, 0) и заканчиваться в точке (1, 1).



def roc_curve_manual(
    y_true: np.ndarray,
    y_score: np.ndarray
) -> Tuple[np.ndarray, np.ndarray, np.ndarray]:
    """Вычисление ROC-кривой путём перебора порогов.

    Параметры:
        y_true  — истинные метки классов, массив из 0 и 1, размерность (n,)
        y_score — оценки вероятности положительного класса, размерность (n,)

    Возвращает:
        Кортеж (fpr, tpr, thresholds), каждый — np.ndarray размерности (k,);
        кривая начинается в (0, 0) и заканчивается в (1, 1)
    """
    thresholds_unique = np.sort(np.unique(y_score))[::-1]

    fpr_list = [0.0]
    tpr_list = [0.0]
    thr_list = [1]

    for t in thresholds_unique:
        y_pred = (y_score >= t).astype(int)
        tp, fp, fn, tn = confusion_matrix_manual(y_true, y_pred)
        pos = tp + fn
        neg = fp + tn
        tpr_list.append(tp / pos if pos > 0 else 0.0)
        fpr_list.append(fp / neg if neg > 0 else 0.0)
        thr_list.append(t)

    if abs(fpr_list[-1] - 1.0) > 1e-6 or abs(tpr_list[-1] - 1.0) > 1e-6:
        fpr_list.append(1.0)
        tpr_list.append(1.0)
        thr_list.append(0)
```

```
    fpr = np.asarray(fpr_list)
    tpr = np.asarray(tpr_list)
    thresholds = np.asarray(thr_list)

    return fpr, tpr, thresholds


def roc_auc_manual(fpr: np.ndarray, tpr: np.ndarray) -> float:
    """Вычисление AUC методом трапеций.

    Параметры:
        fpr — массив значений FPR (ось X ROC-кривой), размерность (k,)
        tpr — массив значений TPR (ось Y ROC-кривой), размерность (k,)

    Возвращает:
        float — площадь под ROC-кривой в диапазоне [0, 1]
    """
    auc = 0.0
    for i in range(len(fpr) - 1):
        auc += (tpr[i] + tpr[i + 1]) / 2.0 * abs(fpr[i + 1] - fpr[i])
    return auc

# ---
# ## Блок 3: Классификация
#
# **Максимум: 30 баллов**
#
# Обучите четыре классификатора на `X_train` / `y_train`, оцените их на
`X_test` / `y_test`. Для подбора гиперпараметров используйте валидационную
выборку `X_val` / `y_val`.


# ### Задание 3.1 — Бейзлайн: логистическая регрессия (4 балла)
#
# Обучите `LogisticRegression` с параметрами по умолчанию. Задайте явно:
# - `random_state=RANDOM_STATE` — для воспроизводимости,
# - `max_iter=1000` — чтобы гарантировать сходимость оптимизатора.
#
# Остальные гиперпараметры **не трогайте** — это точка отсчёта для
сравнения с остальными моделями.
#
```

```
# Сохраните:
# - обученную модель в `logreg`,
# - вероятности положительного класса на тестовой выборке в `y_score_logreg`.
#
# > **Подсказка:** `predict_proba` возвращает массив размерности `(n, 2)` — вам нужен второй столбец (индекс `[:, 1]`).



# Обучаем логистическую регрессию (бейзлайн)
logreg: LogisticRegression = LogisticRegression(
    random_state=RANDOM_STATE,
    max_iter=1000,
)
logreg.fit(X_train, y_train)

# Вероятности положительного класса на тестовой выборке
y_score_logreg: np.ndarray = logreg.predict_proba(X_test)[:, 1]  # размерность (n_test,)

# ### Задание 3.2 — Метод k ближайших соседей (7 баллов)
#
# Обучите `KNeighborsClassifier`. Подберите оптимальное значение `k` по **F1-мере на валидационной выборке** в диапазоне нечётных значений от 1 до 21.
#
# Сохраните:
# - лучшее значение `k` в `best_k`,
# - обученную модель с `best_k` в `knn`,
# - вероятности на тестовой выборке в `y_score_knn`.
#
# > **Подсказка:** метод `predict_proba` у `KNeighborsClassifier` работает только при `algorithm` не `'brute'` для больших датасетов. Если датасет большой — рассмотрите `algorithm='ball_tree'`.



# Подбираем оптимальное k по F1 на валидационной выборке
best_k: int = 1
best_f1: float = -1.0

for k in range(1, 22, 2):
```

```
    knn_candidate = KNeighborsClassifier(n_neighbors=k, algorithm="auto",
    n_jobs=-1)
    knn_candidate.fit(X_train, y_train)
    y_val_pred = knn_candidate.predict(X_val)
    score = f1_manual(y_val, y_val_pred)

    if score > best_f1:
        best_f1 = score
        best_k = k

# Обучаем KNN с best_k на train
knn: KNeighborsClassifier = KNeighborsClassifier(n_neighbors=best_k,
algorithm="auto", n_jobs=-1)
knn.fit(X_train, y_train)

# Вероятности положительного класса на тестовой выборке
y_score_knn: np.ndarray = knn.predict_proba(X_test)[:, 1]  # размерность
(n_test,)

# ### Задание 3.3 — Наивный байесовский классификатор (4 балла)
#
# Обучите `GaussianNB` — вероятностный классификатор, предполагающий
независимость признаков и нормальное распределение каждого из них.
#
# Сохраните:
# - обученную модель в `gnb`,
# - вероятности на тестовой выборке в `y_score_gnb`.
#
# Сравните AUC с логистической регрессией и поразмышляйте: насколько
реалистично предположение о независимости признаков для вашего датасета?



# Обучаем наивный байесовский классификатор
gnb: GaussianNB = GaussianNB()
gnb.fit(X_train, y_train)

# Вероятности положительного класса на тестовой выборке
y_score_gnb: np.ndarray = gnb.predict_proba(X_test)[:, 1]  # размерность
(n_test,)

# ### Задание 3.4 — Метод опорных векторов (7 баллов)
```

#
Обучите `SVC(probability=True)`. Подберите оптимальное значение параметра регуляризации `C` по **F1-мере на валидационной выборке** из сетки `[0.01, 0.1, 1, 10, 100]`.
#
Сохраните:
- лучшее значение в `best_C`,
- обученную модель с `best_C` в `svm`,
- вероятности на тестовой выборке в `y_score_svm`.
#
> **Подсказка:** SVM с ядром RBF и `probability=True` плохо масштабируется — сложность обучения растёт квадратично с числом объектов. На больших датасетах (> 10 000 объектов) время обучения может быть неприемлемым. В таком случае допускается обучать модель на случайной подвыборке обучающих данных — но подумайте, как при этом сохранить соотношение классов. Финальные вероятности `y_score_svm` в любом случае должны вычисляться на полной выборке `X_test`.

```
# Подбираем оптимальное C по F1 на валидационной выборке
grid_C = [0.01, 0.1, 1, 10, 100]

# Для ускорения подбираем C на стратифицированной подвыборке train
X_train_svm, _, y_train_svm, _ = train_test_split(
    X_train,
    y_train,
    train_size=5000,
    random_state=RANDOM_STATE,
    stratify=y_train,
)

best_C: float = grid_C[0]
best_f1: float = -1.0

for C in grid_C:
    svm_candidate = SVC(C=C, probability=True,
    random_state=RANDOM_STATE)
    svm_candidate.fit(X_train_svm, y_train_svm)
    y_val_pred = svm_candidate.predict(X_val)
    score = f1_manual(y_val, y_val_pred)
```

```
    if score > best_f1:
        best_f1 = score
        best_C = C

# Обучаем финальную SVM с best_C
svm: SVC = SVC(C=best_C, probability=True,
random_state=RANDOM_STATE)
svm.fit(X_train_svm, y_train_svm)

# Получаем вероятности на тестовой выборке
y_score_svm: np.ndarray = svm.predict_proba(X_test)[:, 1]  # размерность
(n_test,)

# ### Задание 3.5 — Визуализация ROC и PR кривых (4 балла)
#
# Реализуйте функцию `plot_roc_pr`, которая строит **оба графика** на
одной фигуре:
#
# - **Левый:** ROC-кривые для всех четырёх моделей на одних осях;
пунктирная диагональ (случайный классификатор); AUC каждой модели указан
в легенде.
# - **Правый:** PR-кривые для всех четырёх моделей; горизонтальная линия
на уровне доли положительного класса (случайный классификатор).
#
# Используйте **собственные реализации** `roc_curve_manual` и
`precision_recall_curve_manual` из блока 2.
#
# После построения ответьте в `roc_pr_comment`: какая кривая
информативнее для вашего датасета и почему?



def plot_roc_pr(
    scores_dict: dict,
    y_test: np.ndarray
) -> matplotlib.figure.Figure:
    """Визуализация ROC-кривой и PR-кривой для нескольких моделей.

    Параметры:
        scores_dict — словарь {имя_модели: массив вероятностей на тесте}
        y_test      — истинные метки тестовой выборки
```

```
Возвращает:
    matplotlib.figure.Figure с двумя подграфиками:
    левый — ROC-кривые всех моделей, правый — PR-кривые всех моделей
"""
y_true = np.asarray(y_test)

fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# ROC
for model_name, y_score in scores_dict.items():
    fpr, tpr, _ = roc_curve_manual(y_true, y_score)
    auc = roc_auc_manual(fpr, tpr)
    axes[0].plot(fpr, tpr, linewidth=2, label=f"{model_name} (AUC={auc:.3f})")

axes[0].plot([0, 1], [0, 1], "k--", linewidth=1.5, label="Random")
axes[0].set_title("ROC-кривые")
axes[0].set_xlabel("FPR")
axes[0].set_ylabel("TPR")
axes[0].set_xlim(0, 1)
axes[0].set_ylim(0, 1)
axes[0].grid(alpha=0.3)
axes[0].legend()

# PR
positive_rate = np.mean(y_true)
for model_name, y_score in scores_dict.items():
    precision, recall, _ = precision_recall_curve_manual(y_true, y_score)
    axes[1].plot(recall, precision, linewidth=2, label=model_name)

axes[1].axhline(positive_rate, color="k", linestyle="--", linewidth=1.5,
label=f"Random (p={positive_rate:.3f})")
axes[1].set_title("PR-кривые")
axes[1].set_xlabel("Recall")
axes[1].set_ylabel("Precision")
axes[1].set_xlim(0, 1)
axes[1].set_ylim(0, 1)
axes[1].grid(alpha=0.3)
axes[1].legend()

fig.tight_layout()
return fig
```

```
# Построение графика после обучения всех четырёх моделей
scores_dict = {
    "LogReg": y_score_logreg,
    "KNN": y_score_knn,
    "GaussianNB": y_score_gnb,
    "SVM": y_score_svm,
}
fig_roc_pr: matplotlib.figure.Figure = plot_roc_pr(scores_dict, y_test)

# Интерпретация результатов
roc_pr_comment: str = (
    "PR-кривая информативнее для этого датасета, потому что класс
    просрочки(1) встречается реже, "
    "и PR напрямую отражает компромисс между Precision и Recall именно для
    положительного(1) класса. "
    "ROC-кривая полезна для общей оценки, но при дисбалансе классов она"
    "хуже показывают цену ложноположительных срабатываний."
)



# ### Задание 3.6 — Подбор порога, сводная таблица и матрицы ошибок (8
баллов)
#
# **Часть А. Подбор оптимального порога.**
# Для каждой из четырёх моделей подберите оптимальный порог по **F1-мере
на валидационной выборке**: переберите пороги от 0.05 до 0.95 с шагом 0.05 и
выберите тот, при котором F1 максимально. Сохраните результат в словарь
`thresholds`.
#
# **Часть Б. Сводная таблица метрик.**
# Реализуйте функцию `compare_models`, которая применяет оптимальные
пороги к тестовой выборке и возвращает сводную таблицу метрик.
# Таблица должна содержать строки: `LogReg`, `KNN`, `GaussianNB`, `SVM`
— и столбцы: `Accuracy`, `Precision`, `Recall`, `F1`, `ROC-AUC`.
#
# **Часть В. Визуализация матриц ошибок.**
```

Реализуйте функцию `plot_confusion_matrices`, которая строит матрицы ошибок для всех четырёх моделей на одной фигуре (2×2 подграфика). Используйте оптимальные пороги из `thresholds`. Подпишите оси и отобразите значения TP, FP, FN, TN в каждой ячейке.
#
Используйте **ручную реализацию** `confusion_matrix_manual` из задания 2.1.
#
> **Подсказка:** матрица ошибок наглядно показывает, где именно ошибается модель — какой тип ошибок (FP или FN) преобладает. Сравните матрицы моделей: одинаково ли они ошибаются?

```
def compare_models(
    models_dict: Dict[str, np.ndarray],
    y_test: np.ndarray,
    thresholds: Dict[str, float]
) -> pd.DataFrame:
    """Сводная таблица метрик классификации для нескольких моделей.

    Параметры:
        models_dict — словарь {имя_модели: массив вероятностей на тесте}
        y_test      — истинные метки тестовой выборки
        thresholds  — словарь {имя_модели: оптимальный_порог}

    Возвращает:
        pd.DataFrame со строками по моделям и столбцами:
        Accuracy, Precision, Recall, F1, ROC-AUC
    """
    model_names = ["LogReg", "KNN", "GaussianNB", "SVM"]
    rows = []

    for name in model_names:
        y_score = models_dict[name]
        t = thresholds[name]
        y_pred = (y_score >= t).astype(int)

        acc = accuracy_manual(y_test, y_pred)
        prec = precision_manual(y_test, y_pred)
        rec = recall_manual(y_test, y_pred)
        f1 = f1_manual(y_test, y_pred)
```

```
        fpr, tpr, _ = roc_curve_manual(y_test, y_score)
        auc = roc_auc_manual(fpr, tpr)

        rows.append([acc, prec, rec, f1, auc])

    return pd.DataFrame(
        rows,
        index=model_names,
        columns=["Accuracy", "Precision", "Recall", "F1", "ROC-AUC"],
    )


def plot_confusion_matrices(
    models_dict: Dict[str, np.ndarray],
    y_test: np.ndarray,
    thresholds: Dict[str, float]
) -> matplotlib.figure.Figure:
    """Матрицы ошибок для нескольких моделей на одной фигуре.

    Параметры:
        models_dict — словарь {имя_модели: массив вероятностей на тесте}
        y_test      — истинные метки тестовой выборки
        thresholds  — словарь {имя_модели: оптимальный_порог}

    Возвращает:
        matplotlib.figure.Figure с подграфиками 2×2 — по одному на модель;
        каждый подграфик содержит матрицу ошибок с подписями TP, FP, FN,
        TN
    """
    model_names = ["LogReg", "KNN", "GaussianNB", "SVM"]

    fig, axes = plt.subplots(2, 2, figsize=(12, 10))
    axes = axes.ravel()

    for i, name in enumerate(model_names):
        scores = models_dict[name]
        t = thresholds[name]
        y_pred = (scores >= t).astype(int)

        tp, fp, fn, tn = confusion_matrix_manual(y_test, y_pred)
```

```
        matrix = np.array([[tn, fp], [fn, tp]])

        ax = axes[i]
        im = ax.imshow(matrix, cmap="Reds")
        ax.set_title(name)
        ax.set_xlabel("Предсказанный")
        ax.set_ylabel("Фактический")
        ax.set_xticks([0, 1])
        ax.set_yticks([0, 1])
        ax.set_xticklabels(["0", "1"])
        ax.set_yticklabels(["0", "1"])

        labels = np.array([[f"TN\n{tn}", f"FP\n{fp}"], [f"FN\n{fn}", f"TP\n{tp}"]])
        for r in range(2):
            for c in range(2):
                ax.text(c, r, labels[r, c], ha="center", va="center", color="black",
                fontsize=10)

    fig.tight_layout()
    return fig


# Часть A: подбор оптимальных порогов по F1 на валидационной выборке
threshold_grid = np.linspace(0.05, 0.95, 19)

val_scores: Dict[str, np.ndarray] = {
    "LogReg": logreg.predict_proba(X_val)[:, 1],
    "KNN": knn.predict_proba(X_val)[:, 1],
    "GaussianNB": gnb.predict_proba(X_val)[:, 1],
    "SVM": svm.predict_proba(X_val)[:, 1],
}

thresholds: Dict[str, float] = {}
for model_name, scores in val_scores.items():
    best_t = threshold_grid[0]
    best_f1 = -1.0
    for t in threshold_grid:
        y_pred = (scores >= t).astype(int)
        score = f1_manual(y_val, y_pred)
        if score > best_f1:
            best_f1 = score
```

```
            best_t = t
    thresholds[model_name] = best_t

# Часть Б: сводная таблица на тестовой выборке
models_dict_test = {
    "LogReg": y_score_logreg,
    "KNN": y_score_knn,
    "GaussianNB": y_score_gnb,
    "SVM": y_score_svm,
}
results_df: pd.DataFrame = compare_models(models_dict_test, y_test, thresholds)

# Часть В: матрицы ошибок
fig_confusion: matplotlib.figure.Figure =
plot_confusion_matrices(models_dict_test, y_test, thresholds)

# Отобразите таблицу
if results_df is not None:
    display(results_df.style.highlight_max(axis=0, color='blue'))



# ---
# ## Блок 4: Бонусные задания
#
# **Максимум: 20 бонусных баллов**
#
# Бонусные задания выполняются по желанию и не влияют на получение
основных баллов.
#
# >  **Если вы не планируете выполнять бонусные задания:** удалите
объявления функций и классов из этого блока, а переменным присвойте `None`
или удалите их полностью. Незаполненные заглушки (`raise
NotImplementedError`) вызовут ошибку при запуске тестов.
#



# ###  Задание 4.1 — Работа с дисбалансом классов (10 бонусных баллов)
#
# Исследуйте влияние балансировки классов на качество логистической
регрессии.
#
```

Шаг 1. Визуализируйте распределение классов до и после балансировки с помощью функции `plot_class_distribution`.
#
Шаг 2. Выберите и примените одну из стратегий:
- `SMOTE` из `imblearn.over_sampling` — генерация синтетических примеров миноритарного класса,
- `RandomUnderSampler` из `imblearn.under_sampling` — удаление части примеров мажоритарного класса,
- параметр `class_weight='balanced'` в `LogisticRegression` — взвешивание без изменения выборки.
#
Шаг 3. Обучите `LogisticRegression` на сбалансированных данных и сравните метрики с бейзлайном из задания 3.1. Запишите наблюдения в `balance_comment`: как изменились precision, recall и F1 для миноритарного класса?
#
> **Подсказка:** балансировку применяйте **только к обучающей выборке** — `X_train`, `y_train`. Валидационная и тестовая выборки остаются без изменений.

```
def plot_class_distribution(
    y_before: np.ndarray,
    y_after: np.ndarray
) -> matplotlib.figure.Figure:
    """Визуализация распределения классов до и после балансировки.

    Параметры:
        y_before — целевая переменная до балансировки
        y_after  — целевая переменная после балансировки

    Возвращает:
        matplotlib.figure.Figure с двумя подграфиками (до / после)
    """
    y_before = np.asarray(y_before).astype(int)
    y_after = np.asarray(y_after).astype(int)

    before_counts = pd.Series(y_before).value_counts().sort_index()
    after_counts = pd.Series(y_after).value_counts().sort_index()

    fig, axes = plt.subplots(1, 2, figsize=(12, 4))
```

```
    axes[0].bar(before_counts.index.astype(str), before_counts.values,
    color="tab:blue")
    axes[0].set_title("До балансировки")
    axes[0].set_xlabel("Класс")
    axes[0].set_ylabel("Количество")

    axes[1].bar(after_counts.index.astype(str), after_counts.values,
    color="tab:orange")
    axes[1].set_title("После балансировки")
    axes[1].set_xlabel("Класс")
    axes[1].set_ylabel("Количество")

    fig.tight_layout()
    return fig


# Используем один метод балансировки: class_weight='balanced'
logreg_balanced: LogisticRegression = LogisticRegression(
    random_state=RANDOM_STATE,
    max_iter=1000,
    class_weight="balanced",
)
logreg_balanced.fit(X_train, y_train)
y_score_logreg_balanced: np.ndarray = logreg_balanced.predict_proba(X_test)[:,
1]

# Визуализация распределения классов
fig_balance: matplotlib.figure.Figure = plot_class_distribution(y_train, y_train)

# Подбор порога для balanced-модели по F1 на валидационной выборке
threshold_grid = np.linspace(0.05, 0.95, 19)
y_val_score_balanced = logreg_balanced.predict_proba(X_val)[:, 1]

best_threshold: float = threshold_grid[0]
best_f1_val: float = -1.0
for t in threshold_grid:
    y_val_pred = (y_val_score_balanced >= t).astype(int)
    f1_val = f1_manual(y_val, y_val_pred)
    if f1_val > best_f1_val:
        best_f1_val = f1_val
```

```
        best_threshold = t

# Наблюдения на тесте: baseline vs class_weight='balanced'
y_pred_base = (y_score_logreg >= thresholds["LogReg"]).astype(int)
y_pred_bal = (y_score_logreg_balanced >= best_threshold).astype(int)

base_precision = precision_manual(y_test, y_pred_base)
base_recall = recall_manual(y_test, y_pred_base)
base_f1 = f1_manual(y_test, y_pred_base)

bal_precision = precision_manual(y_test, y_pred_bal)
bal_recall = recall_manual(y_test, y_pred_bal)
bal_f1 = f1_manual(y_test, y_pred_bal)

print(f"Лучший порог для balanced-модели (по F1 на val): {best_threshold:.2f};
F1_val={best_f1_val:.3f}")
print(f"Baseline -> Precision: {base_precision:.3f}, Recall: {base_recall:.3f}, F1:
{base_f1:.3f}")
print(f"Balanced -> Precision: {bal_precision:.3f}, Recall: {bal_recall:.3f}, F1:
{bal_f1:.3f}")

balance_comment: str = (
    "Использован метод class_weight='balanced' с подбором порога по F1 на
    валидации (t=0.55, F1_val=0.534). "
    "На тесте по сравнению с baseline precision вырос с 0.526 до 0.534, "
    "но recall снизился с 0.534 до 0.520, из-за чего итоговая F1 немного снизилась
    с 0.530 до 0.527. "
    "Следовательно, в этой конфигурации balanced-модель не дала прироста по
    F1, "
    "хоть и увеличилась точность положительных предсказаний."
)

# ###  Задание 4.2 — Логистическая регрессия с нуля (10 бонусных баллов)
#
# Реализуйте класс `LogisticRegressionManual`, обучающий модель методом
**полного градиентного спуска** по логарифмической функции потерь с
L2-регуляризацией.
#
# На каждой эпохе градиент вычисляется **по всей обучающей выборке** — не
по одному объекту (стохастический GD) и не по батчу (мини-батчевый GD).
#
```

Функционал качества (отрицательное логарифмическое правдоподобие с регуляризацией):
#
$$Q(\boldsymbol{\theta}, X^\ell) =
-\frac{1}{\ell}\sum_{i=1}^{\ell}\mathscr{L}(\boldsymbol{\theta},
\mathbf{x}_i) + \frac{\tau}{2}\|\boldsymbol{\theta}\|^2$$
#
где функция потерь на одном примере:
#
$$\mathscr{L}(\boldsymbol{\theta}, \mathbf{x}_i) = y_i \log
\sigma(\boldsymbol{\theta}^\top \mathbf{x}_i + \theta_0) + (1 - y_i) \log (1 -
\sigma(\boldsymbol{\theta}^\top \mathbf{x}_i + \theta_0))$$
#
Сигмоида: $\sigma(z) = \dfrac{1}{1 + e^{-z}}$
#
Градиент по $\boldsymbol{\theta}$ (без смещения θ_0):
#
$$\nabla_{\boldsymbol{\theta}} Q = \frac{1}{\ell} X^\top (\hat{y} - y) +
\tau \boldsymbol{\theta}$$
#
Шаг обновления параметров:
#
$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \cdot
\nabla_{\boldsymbol{\theta}} Q$$
#
где η — скорость обучения, τ — коэффициент регуляризации.
#
Требования к классу:
- Метод `fit` должен обновлять `self.theta` и `self.theta_0` на каждой эпохе и сохранять историю значений Q в `self.loss_history`.
- Метод `predict_proba` должен возвращать массив размерности `(n, 2)` — вероятности обоих классов.
- Функция `plot_loss_curve` должна строить график убывания Q по эпохам.
#
После обучения сравните ROC-AUC вашей реализации с `sklearn.linear_model.LogisticRegression`.
#
> **На защите будьте готовы вывести и обосновать приведённые в этом задании формулы.**

```
# TODO: обучите LogisticRegressionManual на X_train, y_train
# Убедитесь, что данные стандартизированы — градиентный спуск чувствителен к масштабу
theta: np.ndarray = None          # размерность (n_features,)
theta_0: float = None
loss_history: np.ndarray = None  # размерность (n_epochs,)

# TODO: постройте кривую обучения
fig_loss: matplotlib.figure.Figure = None

# ---
# ## Финальные проверки
#
# Перед сдачей убедитесь, что:
# - Все ячейки выполнены (`Kernel → Restart & Run All`)
# - Все обязательные переменные определены и не равны `None`
# - Все графики имеют заголовок и подписи осей
# - Текстовые комментарии (`eda_comment`, `preprocessing_comment`, `roc_pr_comment`) заполнены


required_vars = [
    # Блок 0
    'Student_ID', 'DATASET_NAME', 'UCI_ID', 'TARGET_COL',
    # Блок 1
    'df', 'fig_eda', 'eda_comment',
    'df_base',
    'df_processed', 'preprocessing_comment',
    'X_train', 'X_val', 'X_test', 'y_train', 'y_val', 'y_test',
    # Блок 2
    'confusion_matrix_manual',
    'accuracy_manual', 'precision_manual', 'recall_manual', 'f1_manual',
    'precision_recall_curve_manual',
    'roc_curve_manual', 'roc_auc_manual',
    # Блок 3
    'logreg', 'y_score_logreg',
    'knn', 'y_score_knn', 'best_k',
    'gnb', 'y_score_gnb',
    'svm', 'y_score_svm', 'best_C',
```

```
    'fig_roc_pr', 'roc_pr_comment',
    'thresholds', 'compare_models', 'results_df',
    'plot_confusion_matrices', 'fig_confusion',
]

print('=' * 60)
print('ПРОВЕРКА ОБЯЗАТЕЛЬНЫХ ПЕРЕМЕННЫХ:')
print('=' * 60)
all_ok = True
for var in required_vars:
    value = globals().get(var)
    if value is not None:
        print(f'  {var}')
    else:
        print(f'  {var} — НЕ ОПРЕДЕЛЕНА!')
        all_ok = False

bonus_vars = [
    # Блок 4 (бонус)
    'logreg_balanced', 'y_score_logreg_balanced',
    'fig_balance', 'balance_comment',
    'LogisticRegressionManual', 'theta', 'theta_0',
    'loss_history', 'fig_loss',
]

print()
print('=' * 60)
print('ПРОВЕРКА БОНУСНЫХ ПЕРЕМЕННЫХ:')
print('=' * 60)
for var in bonus_vars:
    value = globals().get(var)
    if value is not None:
        print(f'  {var}')
    else:
        print(f'  {var} — не выполнено (бонус)')

print()
if all_ok:
    print(' Все обязательные переменные определены. Работа готова к сдаче!')
else:
    print('  Есть незаполненные обязательные переменные. Проверьте код.')
```

5) Выводы

В процессе выполнения лабораторной работы были получены практические навыки по реализации ключевых метрик классификации и применению алгоритмов на датасетах с дисбалансом классов